SQL Based Data Analysis Project

FRAUD MONITORING SYSTEM

Credit/Debit Card Analysis and Fraud Detection

Presented by-
JIGISHA ABHIJIT KARNIK

INTRODUCTION

The **Fraud Monitoring System** is a structured SQL-based project designed to detect and analyze suspicious financial transactions using relational databases. The system stores customer profiles, payment card details, transaction logs, and fraud alerts in a connected schema, allowing efficient retrieval and investigation of high-risk activity.

Through SQL queries, aggregation, window functions, joins, and sub queries, the database identifies unusual spending behavior such as high-value transactions, international purchases, ATM withdrawal bursts, and activity from blocked cards. These patterns are flagged into a dedicated fraud alert table where they can be examined, monitored, and resolved.

This project simulates how real banking systems analyze digital payment activity, making it useful for risk analysis, cybersecurity training, and financial transaction study. It provides a clear foundation for future integration with dashboards, automation models, and machine learning-based fraud detection engines.

INDEX

1 INTRODUCTION

2 PROJECT OBJECTIVE

3 ER DIAGRAM

4 TABLE STRUCTURE

- CUSTOMERS_PROFILE
- PAYMENT_CARDS
- TRANSACTION_LOGS
- FRAUD_ALERTS

5 CONTENT OF TABLE

6 SQL ANALYSIS & RESULTS

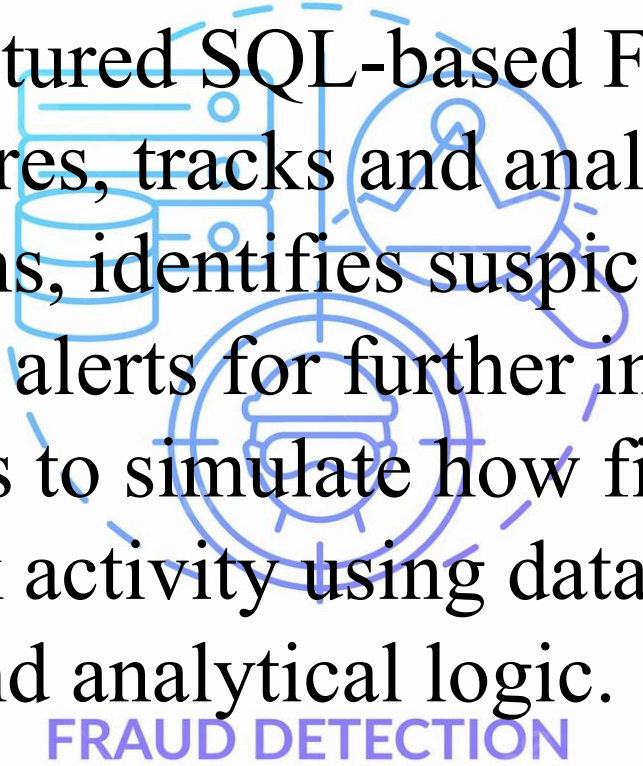
- BASIC QUERY & ANALYSIS
- WINDOWS FUNCTION
- JOINS
- SUB QUERIES

7 FINAL CONCLUSION

8 THANK YOU

PROJECT OBJECTIVE

To build a structured SQL-based Fraud Monitoring System that stores, tracks and analyzes debit/credit card transactions, identifies suspicious patterns and generates fraud alerts for further investigation.
The objective is to simulate how financial institutions detect high-risk activity using database queries, relationships and analytical logic.



ER DIAGRAM

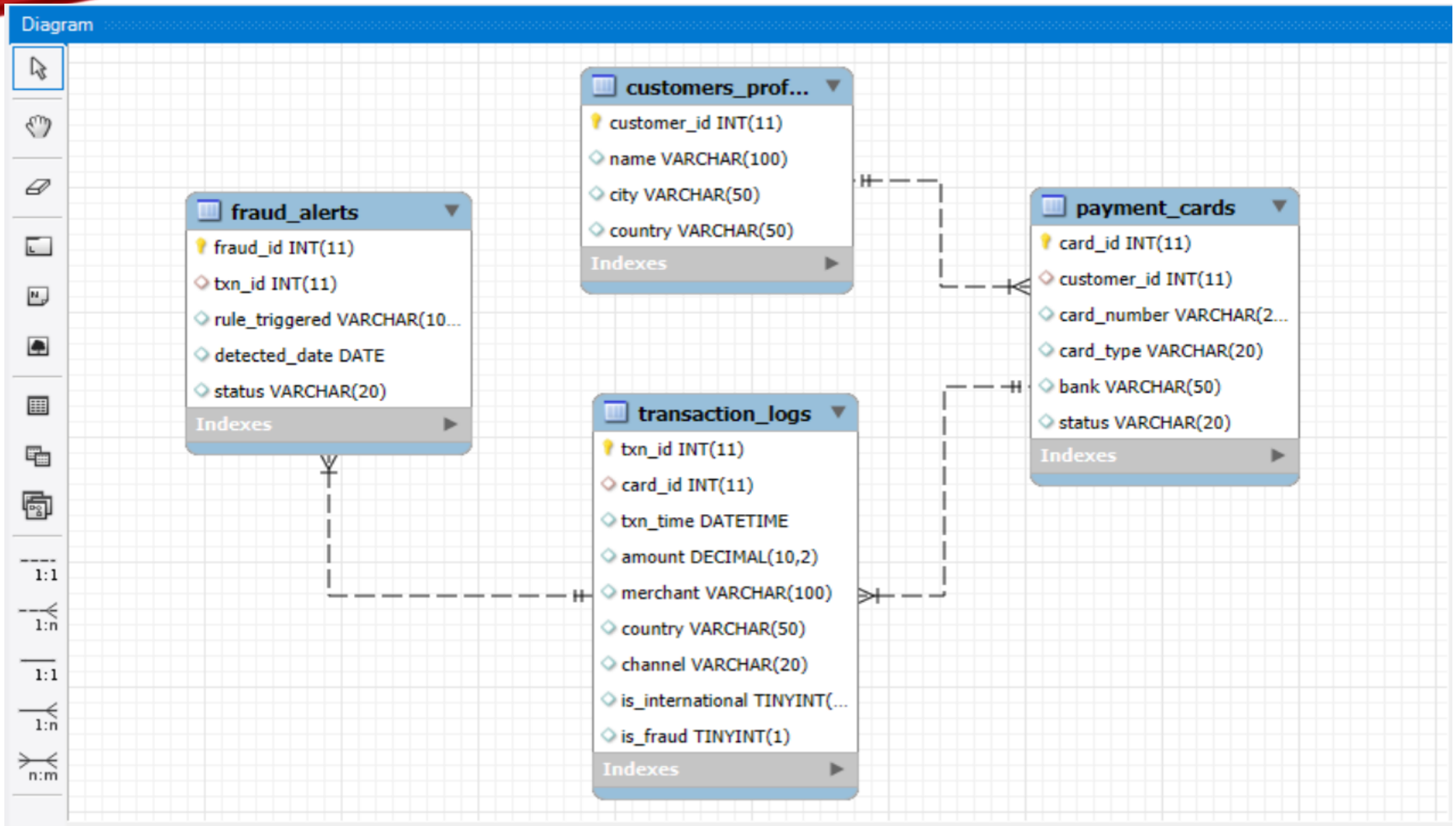


Diagram
customers_prof...
customer_id INT(11)
name VARCHAR(100)
city VARCHAR(50)
country VARCHAR(50)
Indexes
fraud_alerts
fraud_id INT(11)
txn_id INT(11)
rule_triggered VARCHAR(10...
detected_date DATE
status VARCHAR(20)
Indexes
payment_cards
card_id INT(11)
customer_id INT(11)
card_number VARCHAR(2...
card_type VARCHAR(20)
bank VARCHAR(50)
status VARCHAR(20)
Indexes
transaction_logs
txn_id INT(11)
card_id INT(11)
txn_time DATETIME
amount DECIMAL(10,2)
merchant VARCHAR(100)
country VARCHAR(50)
channel VARCHAR(20)
is_international TINYINT(...
is_fraud TINYINT(1)
Indexes
1:1
1:n
1:1
1:n
n:m

An ER Diagram (Entity–Relationship Diagram) is a conceptual model used in database design to represent real-world entities and the logical associations among them. It defines the structure of a database by illustrating entities, attributes, primary keys, and relationships — helping in planning, normalization, and database implementation.

In this project, the **ER Diagram** represents the complete structure of the Fraud Monitoring System database.

It visually maps how **customers**, **their payment cards**, **their transaction activities**, and **fraud alerts** are interconnected.

Through entities like *customers_profile*, *payment_cards*, *transaction_logs*, and *fraud_alerts*, the ER model shows:

✓ what data each table stores,

✓ primary–foreign key links between them, and

✓ how a single customer can own multiple cards and generate multiple transactions, some of which may trigger fraud alerts.

In short, the ER Diagram acts as a **visual blueprint of the fraud detection workflow**, from customer identity to card activity and final fraud flagging.

STRUCTURE OF TABLE

★ *CUSTOMERS_PROFILE TABLE*

```
463 •   DESCRIBE customers_profile;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

Field	Type	Null	Key	Default	Extra
customer_id	int(11)	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
city	varchar(50)	YES		NULL	
country	varchar(50)	YES		NULL	

★ *PAYMENTS_CARD TABLE*

```
465 •   DESCRIBE payment_cards;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

Field	Type	Null	Key	Default	Extra
card_id	int(11)	NO	PRI	NULL	auto_increment
customer_id	int(11)	YES	MUL	NULL	
card_number	varchar(20)	YES		NULL	
card_type	varchar(20)	YES		NULL	
bank	varchar(50)	YES		NULL	
status	varchar(20)	YES		NULL	

STRUCTURE OF TABLE

★ TRANSACTION_LOGS TABLE

```
469 •   DESCRIBE transaction_logs;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

Field	Type	Null	Key	Default	Extra
txn_id	int(11)	NO	PRI	NULL	auto_increment
card_id	int(11)	YES	MUL	NULL	
txn_time	datetime	YES		NULL	
amount	decimal(10,2)	YES		NULL	
merchant	varchar(100)	YES		NULL	
country	varchar(50)	YES		NULL	
channel	varchar(20)	YES		NULL	
is_international	tinyint(1)	YES		NULL	
is_fraud	tinyint(1)	YES		NULL	

★ FRAUD_ALERTS TABLE

```
467 •   DESCRIBE fraud_alerts;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
fraud_id	int(11)	NO	PRI	NULL	auto_increment
txn_id	int(11)	YES	MUL	NULL	
rule_triggered	varchar(100)	YES		NULL	
detected_date	date	YES		NULL	
status	varchar(20)	YES		Open	

CONTENT OF TABLE

CUSTOMERS_PROFILE TABLE

```
471 •   SELECT * FROM customers_profile;
```

Result Grid | Filter Rows: | Edit:

customer_id	name	city	country
1	Aarav Mehta	Mumbai	India
2	Riya Kapoor	Delhi	India
3	Kabir Shah	Pune	India
4	Simran Arora	Bangalore	India
5	Rehan Siddiqui	Lucknow	India
6	Zara Khan	Hyderabad	India
7	Mahi Verma	Chennai	India
8	Yash Malhotra	Jaipur	India
9	Anaya Joshi	Nagpur	India
10	Devansh Shetty	Goa	India
11	Mira Fernandes	Kochi	India
12	Aryan Patel	Ahmedabad	India
13	Neel Bhatt	Surat	India
14	Tanya Oberoi	Indore	India
15	Samar Khurana	Noida	India
16	Aisha Sheikh	Dubai	UAE

Result Grid | Filter Rows: | Edit:

customer_id	name	city	country
16	Aisha Sheikh	Dubai	UAE
17	Rayyan Malik	Doha	Qatar
18	Huda Rahman	London	UK
19	Jason Miller	New York	USA
20	Sophia Lewis	Toronto	Canada
21	Ethan Walker	Sydney	Australia
22	Kim Hana	Seoul	South Korea
23	Hiro Tanaka	Tokyo	Japan
24	Liam Scott	Berlin	Germany
25	Noah Adams	Paris	France
26	Oliver White	Rome	Italy
27	Lucas Gray	Madrid	Spain
28	Emma Green	Melbourne	Australia
29	Daniel King	Singapore	Singapore
30	Ava Taylor	Cape Town	South Africa
NULL	NULL	NULL	NULL

CONTENT OF TABLE

- ***PAYMENT_CARDS TABLE***

```
473 •  SELECT * FROM payment_cards;
```

Result Grid | Filter Rows: | Edit: | Export

card_id	customer_id	card_number	card_type	bank	status
1	1	4111-XXXX-0011	Credit	HDFC Bank	Active
2	1	5222-XXXX-0012	Debit	HDFC Bank	Active
3	2	4333-XXXX-0021	Credit	SBI Bank	Active
4	3	4444-XXXX-0031	Credit	ICICI Bank	Active
5	4	4555-XXXX-0041	Credit	Axis Bank	Blocked
6	5	4666-XXXX-0051	Debit	SBI Bank	Active
7	6	4777-XXXX-0061	Credit	Kotak Bank	Active
8	7	4888-XXXX-0071	Debit	Yes Bank	Active
9	8	4999-XXXX-0081	Credit	Citi Bank	Active
10	9	5000-XXXX-0091	Debit	HSBC Bank	Active
11	10	5111-XXXX-0101	Credit	Axis Bank	Active
12	11	5222-XXXX-0111	Credit	ICICI Bank	Blocked
13	11	5333-XXXX-0112	Debit	ICICI Bank	Active
14	12	5444-XXXX-0121	Credit	SBI Bank	Active
15	13	5555-XXXX-0131	Debit	HDFC Bank	Active
16	14	5666-XXXX-0141	Credit	Kotak Bank	Active

Result Grid | Filter Rows: | Edit: | Expor

card_id	customer_id	card_number	card_type	bank	status
16	14	5666-XXXX-0141	Credit	Kotak Bank	Active
17	14	5777-XXXX-0142	Debit	Kotak Bank	Active
18	15	5888-XXXX-0151	Credit	Axis Bank	Active
19	16	5999-XXXX-0161	Credit	Emirates ...	Active
20	17	6000-XXXX-0171	Credit	QNB Bank	Blocked
21	18	6111-XXXX-0181	Debit	Barclays	Active
22	19	6222-XXXX-0191	Credit	Chase Bank	Active
23	20	6333-XXXX-0201	Debit	RBC Bank	Active
24	21	6444-XXXX-0211	Credit	Commonw...	Active
25	21	6555-XXXX-0212	Debit	Commonw...	Active
26	22	6666-XXXX-0221	Debit	Korea Exc...	Active
27	23	6777-XXXX-0231	Credit	Mizuho Bank	Active
28	23	6888-XXXX-0232	Debit	Mizuho Bank	Active
29	24	6999-XXXX-0241	Credit	Deutsche ...	Active
30	24	7000-XXXX-0242	Debit	Deutsche ...	Blocked
31	25	7111-XXXX-0251	Credit	BNP Paribas	Active

CONTENT OF TABLE

TRANSACTION_LOGS TABLE

```
475 •   SELECT * FROM transaction_logs;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

txn_id	card_id	txn_time	amount	merchant	country	channel	is_international	is_fraud
1	1	2025-01-01 10:15:00	1200.00	Amazon India	India	ONLINE	0	0
2	1	2025-01-03 16:40:00	550.00	Swiggy	India	ONLINE	0	0
3	2	2025-01-04 11:00:00	2400.00	DMart	India	POS	0	0
4	3	2025-01-05 18:25:00	900.00	Zomato	India	ONLINE	0	0
5	4	2025-01-06 10:50:00	3200.00	Reliance Trends	India	POS	0	0
6	5	2025-01-06 19:45:00	450.00	Blinkit	India	ONLINE	0	0
7	6	2025-01-07 14:12:00	1800.00	Croma	India	POS	0	0
8	7	2025-01-08 20:30:00	600.00	McDonalds	India	POS	0	0
9	8	2025-01-09 13:55:00	1100.00	Nykaa	India	ONLINE	0	0
10	9	2025-01-10 17:15:00	1750.00	Amazon India	India	ONLINE	0	0
11	10	2025-01-11 15:32:00	900.00	Dominos	India	ONLINE	0	0
12	11	2025-01-12 12:40:00	670.00	Pizza Hut	India	ONLINE	0	0
13	12	2025-01-12 21:35:00	320.00	Uber India	India	ONLINE	0	0
14	13	2025-01-13 09:22:00	1400.00	BigBazaar	India	POS	0	0
15	14	2025-01-14 18:30:00	600.00	Starbucks	India	POS	0	0
16	15	2025-01-15 14:10:00	2400.00	DMart	India	POS	0	0

CONTENT OF TABLE

FRAUD_ALERTS TABLE

```
477 •   SELECT * FROM fraud_alerts;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

fraud_id	txn_id	rule_triggered	detected_date	status
25	61	High International Transaction	2025-03-01	Open
26	62	Repeated Foreign High Spend	2025-03-01	Investigating
27	63	Crypto Purchase Suspected	2025-03-02	Open
28	64	Unusual High Amount	2025-03-02	Open
29	65	Foreign Merchant Purchase Risk	2025-03-03	Open
30	45	Blocked Card Attempted	2025-02-05	Open
31	46	Repeated Blocked Card Usage	2025-02-05	Investigating
32	70	Odd-Hour Transaction Activity	2025-03-05	Open
33	72	Night Transaction Spike	2025-03-06	Investigating
34	33	Multiple ATM Withdrawals Detected	2025-02-01	Investigating
35	34	ATM Withdrawal Spike	2025-02-01	Open
36	38	Micro-Transaction Pattern	2025-02-03	Resolved
37	39	Card Testing Suspected	2025-02-03	Open
38	74	Location Shift India→Dubai/UK	2025-03-07	Investigating
39	77	Different Country Same Day	2025-03-08	Open
40	82	NFT Purchase Alert	2025-03-08	Open

SQL ANALYSIS & RESULTS

BASIC QUERIES AND ANALYSIS

Basic SQL queries help us retrieve fundamental information from the database such as total number of users, number of cards issued, count of transactions, fraud vs non-fraud volume, highest spending card, and top merchants.
These queries provide the **first level of understanding** of how the system behaves, how money flows through the cards, and where abnormal patterns begin to appear.

OUTPUT.

Q1) Show total transactions done by each card.

```
SELECT
    card_id,
    COUNT(txn_id) AS total_transactions
FROM transaction_logs
GROUP BY card_id
ORDER BY total_transactions DESC;
```

INSIGHTS:

This query highlights which cards are most actively used.

Cards with unusually high activity count may require monitoring —

frequent usage can indicate risk or misuse, especially if recent

Result Grid | Filter Rows:

card_id	total_transactions
5	5
9	5
17	5
21	4
22	4
11	4
28	4
14	4
30	4
15	4
4	3
1	3
2	3
20	2
6	2
7	2
23	2
24	2
25	2
10	2
26	2
27	2
12	2
13	2
29	2
16	2
18	2
3	2
19	2
8	1
32	1
33	1
34	1
36	1
38	1

Q2) Calculate the number of fraud vs non-fraud transactions

```sql
SELECT
    is_fraud,
    COUNT(*) AS transaction_count
FROM transaction_logs
GROUP BY is_fraud;
```

OUTPUT

Result Grid | Filter Rows:

is_fraud	transaction_count
0	45
1	45

INSIGHTS:

Shows fraud severity in your system.
A higher fraud count = high-risk environment.
Balanced fraud/non-fraud ratio is ideal for study analysis.

Q3) Find top 5 merchants with highest fraud occurrences

```sql
SELECT merchant, COUNT(*) AS fraud_count
FROM transaction_logs
WHERE is_fraud = 1
GROUP BY merchant
ORDER BY fraud_count DESC
LIMIT 5;
```

OUTPUT

Result Grid | Filter Rows:

merchant	fraud_count
ATM Withdrawal	3
Electronics Store	2
Fashion Hub	1
Blinkit	1
ParisFashion	1

INSIGHTS:

This result highlights the merchants where fraud occurs most frequently.
ATM withdrawals show the **highest number of fraud attempts**, indicating a high-risk zone — possibly due to stolen cards or unauthorized PIN access

WINDOWS FUNCTIONS

Window Functions in SQL allow us to perform calculations **across a set of rows without grouping them into a single result.**
Unlike normal GROUP BY, window functions keep each row visible while also allowing analytical operations like running totals, ranking, and moving averages
Window functions turn raw transaction logs into insights — perfect for fraud scoring, risk trend spotting, and priority case escalation.

Q1. Rank transactions by highest amount (fraud risk index)

```sql
SELECT
    txn_id,
    card_id,
    amount,
    RANK() OVER (ORDER BY amount DESC) AS risk_rank
FROM transaction_logs;
```

Insight:
Transactions with rank **1–5 are highest risk**, especially if marked as fraud.
Helps prioritize which transactions to review or freeze first.

Result Grid | Filter Rows:

txn_id	card_id	amount	risk_rank
89	36	144900.00	1
81	26	139000.00	2
73	18	128000.00	3
86	32	127500.00	4
78	23	120000.00	5
70	14	115000.00	6
84	29	113000.00	7
66	9	102500.00	8
90	38	102200.00	9
83	28	99000.00	10
75	20	97000.00	11
85	30	94000.00	12
72	17	93500.00	13
62	3	92000.00	14
69	13	91000.00	15
79	24	88000.00	16
40	17	87500.00	17
88	34	86000.00	18
65	7	85000.00	19
74	19	82000.00	20
61	1	78000.00	21
80	25	77000.00	22
87	33	74000.00	23
68	12	72000.00	24
64	6	68000.00	25
77	22	65500.00	26
82	27	64500.00	27
71	16	64000.00	28
67	10	56000.00	29
76	21	54000.00	30
60	14	50000.00	31
63	5	45000.00	32
54	2	45000.00	32
47	22	24000.00	34
46	22	20000.00	35
50	9	18200.00	36
45	11	16200.00	37
52	21	12400.00	38
35	28	9100.00	39
34	28	8900.00	40
33	30	5200.00	41
32	30	5100.00	42
31	30	5000.00	43
5	4	3200.00	44
29	28	2900.00	45

Q2. Running total spend per card (usage surge monitoring).

```
SELECT
    card_id,
    txn_time,
    amount,
    SUM(amount) OVER (PARTITION BY card_id ORDER BY txn_time) AS cumulative_spending
FROM transaction_logs;
```

card_id	txn_time	amount	cumulative_spending
5	2025-03-02 01:30:00	45000.00	45595.00
6	2025-01-07 14:12:00	1800.00	1800.00
6	2025-03-02 23:40:00	68000.00	69800.00
7	2025-01-08 20:30:00	600.00	600.00
7	2025-03-03 00:14:00	85000.00	85600.00
8	2025-01-09 13:55:00	1100.00	1100.00
9	2025-01-10 17:15:00	1750.00	1750.00
9	2025-02-08 17:35:00	550.00	2300.00
9	2025-02-08 17:50:00	799.00	3099.00
9	2025-02-08 18:10:00	18200.00	21299.00
9	2025-03-03 03:10:00	102500.00	123799.00
10	2025-01-11 15:32:00	900.00	900.00
10	2025-03-03 23:50:00	56000.00	56900.00
11	2025-01-12 12:40:00	670.00	670.00
11	2025-02-06 20:05:00	1500.00	2170.00

OUTPUT

Result Grid | Filter Rows: | Export:

card_id	txn_time	amount	cumulative_spending
1	2025-01-01 10:15:00	1200.00	1200.00
1	2025-01-03 16:40:00	550.00	1750.00
1	2025-03-01 02:45:00	78000.00	79750.00
2	2025-01-04 11:00:00	2400.00	2400.00
2	2025-02-10 15:19:00	380.00	2780.00
2	2025-02-10 23:51:00	45000.00	47780.00
3	2025-01-05 18:25:00	900.00	900.00
3	2025-03-01 03:12:00	92000.00	92900.00
4	2025-01-06 10:50:00	3200.00	3200.00
4	2025-02-05 18:34:00	320.00	3520.00
4	2025-02-05 18:50:00	2100.00	5620.00
5	2025-01-06 19:45:00	450.00	450.00
5	2025-02-12 10:01:00	45.00	495.00
5	2025-02-12 10:02:00	51.00	546.00
5	2025-02-12 10:05:00	49.00	595.00

card_id	txn_time	amount	cumulative_spending
11	2025-02-06 20:08:00	1450.00	3620.00
11	2025-02-06 20:14:00	16200.00	19820.00
12	2025-01-12 21:35:00	320.00	320.00
12	2025-03-04 04:55:00	72000.00	72320.00
13	2025-01-13 09:22:00	1400.00	1400.00
13	2025-03-04 02:42:00	91000.00	92400.00
14	2025-01-14 18:30:00	600.00	600.00
14	2025-02-13 09:15:00	2200.00	2800.00
14	2025-02-13 09:40:00	50000.00	52800.00
14	2025-03-05 00:05:00	115000.00	167800.00
15	2025-01-15 14:10:00	2400.00	2400.00
15	2025-02-03 09:10:00	99.00	2499.00
15	2025-02-03 09:12:00	110.00	2609.00
15	2025-02-03 09:14:00	95.00	2704.00
16	2025-01-16 20:45:00	820.00	820.00

card_id	txn_time	amount	cumulative_spending
28	2025-01-27 19:58:00	2900.00	2900.00
28	2025-02-02 01:40:00	8900.00	11800.00
28	2025-02-02 01:48:00	9100.00	20900.00
28	2025-03-11 03:13:00	99000.00	119900.00
29	2025-01-28 11:30:00	750.00	750.00
29	2025-03-11 04:42:00	113000.00	113750.00
30	2025-02-01 22:15:00	5000.00	5000.00
30	2025-02-01 22:18:00	5100.00	10100.00
30	2025-02-01 22:22:00	5200.00	15300.00
30	2025-03-12 00:50:00	94000.00	109300.00
32	2025-03-12 03:28:00	127500.00	127500.00
33	2025-03-13 02:10:00	74000.00	74000.00
34	2025-03-13 23:55:00	86000.00	86000.00
36	2025-03-14 01:22:00	144900.00	144900.00
38	2025-03-14 03:55:00	102200.00	102200.00

Insight:
Shows how spending builds over time per card.
If a card's cumulative value jumps sharply, it may indicate sudden fraud bursts.

Q3. Compare each transaction with average spending of that same card

```
SELECT
    card_id,
    txn_id,
    amount,
    AVG(amount) OVER (PARTITION BY card_id) AS avg_card_spend
FROM transaction_logs;
```

OUTPUT

Result Grid | Filter Rows:

card_id	txn_id	amount	avg_card_spend
1	61	78000.00	26583.333333
1	1	1200.00	26583.333333
1	2	550.00	26583.333333
2	3	2400.00	15926.666667
2	53	380.00	15926.666667
2	54	45000.00	15926.666667
3	4	900.00	46450.000000
3	62	92000.00	46450.000000
4	5	3200.00	1873.333333
4	41	320.00	1873.333333
4	42	2100.00	1873.333333
5	6	450.00	9119.000000
5	63	45000.00	9119.000000
5	56	45.00	9119.000000
5	57	51.00	9119.000000

Result Grid | Filter Rows:

card_id	txn_id	amount	avg_card_spend
28	35	9100.00	29975.000000
28	83	99000.00	29975.000000
28	29	2900.00	29975.000000
28	34	8900.00	29975.000000
29	84	113000.00	56875.000000
29	30	750.00	56875.000000
30	85	94000.00	27325.000000
30	31	5000.00	27325.000000
30	32	5100.00	27325.000000
30	33	5200.00	27325.000000
32	86	127500.00	127500.000000
33	87	74000.00	74000.000000
34	88	86000.00	86000.000000
36	89	144900.00	144900.000000
38	90	102200.00	102200.000000

INSIGHTS:
Helps detect transactions **far above the normal spending level**.
If a transaction is much higher than avg_card_spend, it's a red-flag candidate.

Q4. Rank highest 10 transactions by amount

```sql
SELECT
    txn_id, card_id, amount,
    RANK() OVER (ORDER BY amount DESC) AS risk_rank
FROM transaction_logs
LIMIT 10;
```

OUTPUT

txn_id	card_id	amount	risk_rank
89	36	144900.00	1
81	26	139000.00	2
73	18	128000.00	3
86	32	127500.00	4
78	23	120000.00	5
70	14	115000.00	6
84	29	113000.00	7
66	9	102500.00	8
90	38	102200.00	9
83	28	99000.00	10

INSIGHTS:

Ranking transactions by amount allows us to focus on the biggest risks first.

It helps banks investigate high-value fraud events quickly and efficiently.

JOINS

In SQL, a JOIN clause is used to combine rows from two or more tables based on a related column between them. This allows the retrieval of data that is distributed across multiple tables and the establishment of relationships between them within a single result set.

TYPES OF JOINS:

1. **INNER JOIN (or simply JOIN):** Returns only the rows where there is a match in both tables based on the join condition.

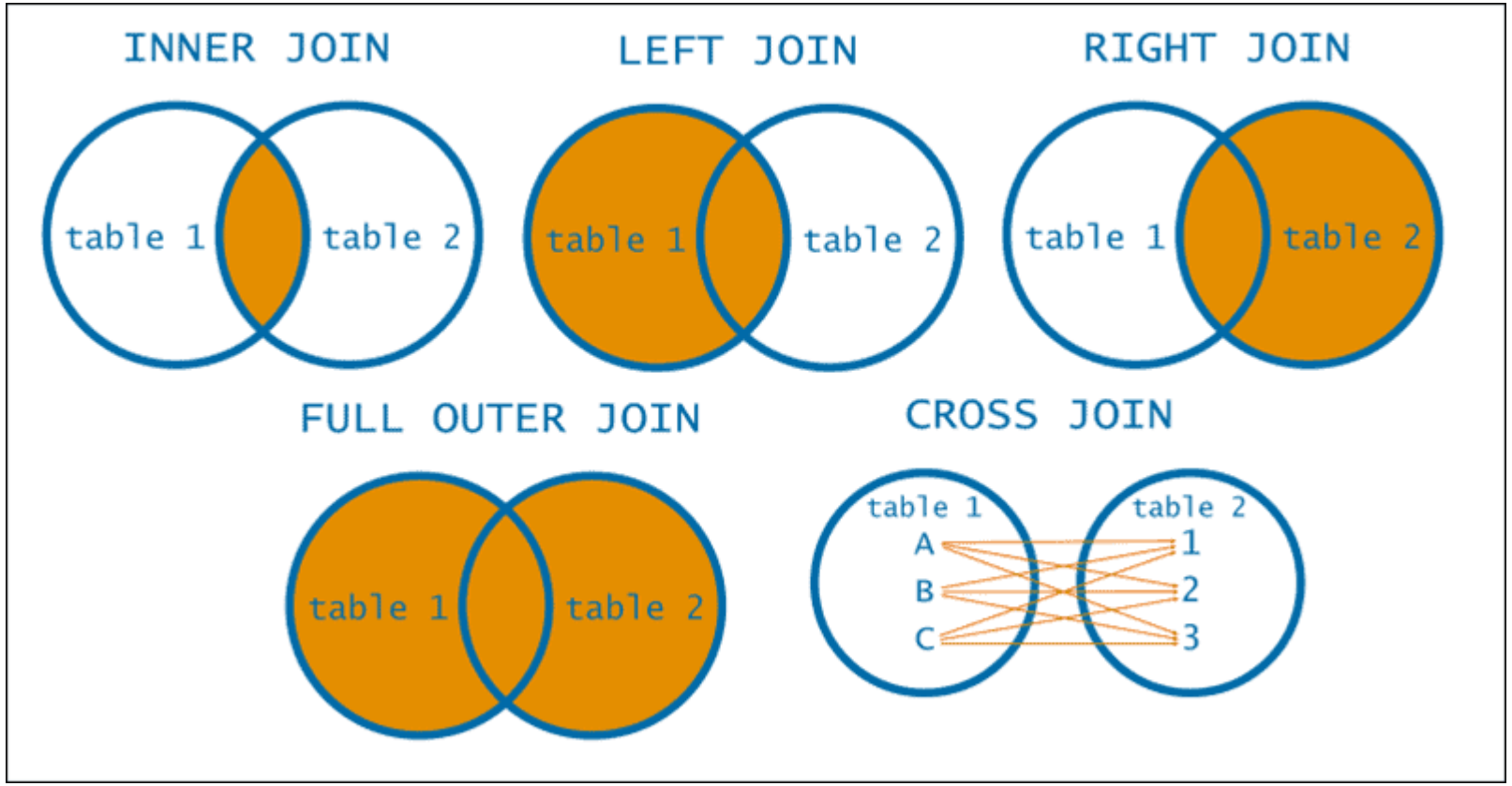
2. **LEFT JOIN (or LEFT OUTER JOIN):** Returns all rows from the left table, and the matching rows from the right table. If no match is found in the right table, NULL values are returned for the right table's columns

RIGHT JOIN (or RIGHT OUTER JOIN): Returns all rows from the right table, and the matching rows from the left table. If no match is found in the left table, NULL values are returned for the left table's columns

FULL JOIN (or FULL OUTER JOIN): Returns all rows when there is a match in either the left or right table. If a row in one table has no match in the other, NULL values are returned for the columns of the non-matching table

CROSS JOIN: Returns the Cartesian product of the two tables, meaning every row from the first table is combined with every row from the second table. This type of join does not require a join condition

TYPES OF JOINS DIAGRAM



Q1. Compare Fraud Count Between Debit & Credit Cards

```sql
SELECT
    pc.card_type,
    COUNT(tl.txn_id) AS total_transactions,
    SUM(tl.is_fraud) AS fraud_transactions
FROM payment_cards pc
JOIN transaction_logs tl ON pc.card_id = tl.card_id
GROUP BY pc.card_type;
```

OUTPUT

Result Grid | Filter Rows: | Exp

card_type	total_transactions	fraud_transactions
Credit	52	26
Debit	38	19

INSIGHTS:

This comparison helps identify which card type needs stronger monitoring and security measures.
The card type with higher fraud traffic is the primary risk zone and should be prioritized for fraud prevention.

Q2. Show fraud alerts with customer name & card used

```sql
SELECT
    fa.fraud_id,
    cp.name AS customer_name,
    pc.card_type,
    tl.amount,
    fa.detected_date
FROM fraud_alerts fa
JOIN transaction_logs tl ON fa.txn_id = tl.txn_id
JOIN payment_cards pc ON tl.card_id = pc.card_id
JOIN customers_profile cp ON pc.customer_id = cp.customer_id
ORDER BY fa.fraud_id
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

fraud_id	customer_name	card_type	amount	detected_date
25	Aarav Mehta	Credit	78000.00	2025-03-01
26	Riya Kapoor	Credit	92000.00	2025-03-01
27	Simran Arora	Credit	45000.00	2025-03-02
28	Rehan Siddiqui	Debit	68000.00	2025-03-02
29	Zara Khan	Credit	85000.00	2025-03-03

Insight:
Shows *who* faced fraud, *which card type* was used and *how much* was lost — helping detect **pattern in victims + card preference used in fraud**.

Q3. List customers who made international fraud transactions

```sql
SELECT
    cp.name,
    pc.card_type,
    tl.amount,
    tl.country
FROM customers_profile cp
JOIN payment_cards pc ON cp.customer_id = pc.customer_id
JOIN transaction_logs tl ON pc.card_id = tl.card_id
WHERE tl.is_fraud = 1 AND tl.is_international = 1
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	card_type	amount	country
Aarav Mehta	Credit	78000.00	Singapore
Riya Kapoor	Credit	92000.00	UK
Simran Arora	Credit	45000.00	USA
Rehan Siddiqui	Debit	68000.00	UAE
Zara Khan	Credit	85000.00	Japan

Insight:
Helps identify **customers affected by overseas fraud**, which is usually high-risk and high-value. Useful for alerting bank systems to trigger **geo-risk warnings**

Q4. Show All Fraud Transactions with Card Details

```sql
SELECT
    cp.name AS customer_name,
    pc.card_number,
    pc.card_type,
    tl.amount,
    tl.country,
    tl.txn_time
FROM transaction_logs tl
JOIN payment_cards pc ON tl.card_id = pc.card_id
JOIN customers_profile cp ON pc.customer_id = cp.customer_id
WHERE tl.is_fraud = 1
ORDER BY tl.amount DESC LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows

customer_name	card_number	card_type	amount	country	txn_time
Emma Green	7666-XXXX-0281	Credit	144900.00	Singapore	2025-03-14 01:22:00
Kim Hana	6666-XXXX-0221	Debit	139000.00	Italy	2025-03-10 01:02:00
Samar Khurana	5888-XXXX-0151	Credit	128000.00	USA	2025-03-06 02:05:00
Oliver White	7222-XXXX-0261	Debit	127500.00	USA	2025-03-12 03:28:00
Sophia Lewis	6333-XXXX-0201	Debit	120000.00	Canada	2025-03-08 23:59:00

INSIGHTS:

This output highlights the **highest value fraudulent transactions** along with the customers, card type, country, and time of occurrence.

Large-value transactions are critical because even a single successful fraud attempt can cause heavy financial loss.

SUB QUERIES

A subquery (also known as an inner query or nested query) is a complete SQL query nested inside another SQL statement, such as a SELECT, INSERT, UPDATE, or DELETE.
The primary purpose of a subquery is to return data that will be used by the main, or outer, query to complete its operation. The inner query runs first, and its result is then used by the outer query to filter rows, provide a calculated value, or act as a temporary table.

Key Characteristics:
Enclosed in Parentheses: A subquery must always be enclosed within parentheses ().
Execution Order: Generally, the inner subquery executes first, and its result is passed to the outer query.
Placement: Subqueries can be used in various clauses of a SQL statement, including SELECT, FROM, WHERE, and HAVING.
Operators: They are often used with comparison operators (=, <, >, etc.) or multiple-row operators (IN, NOT IN, ANY, ALL, EXISTS, NOT EXISTS)

Types of Subqueries

Subqueries are typically categorized by the type of data they return and how they interact with the outer query:

1. Single-Row Subquery: Returns exactly one row and one column (a single value). These are used with single-value comparison operators like =, >, or <.
2.Multi-Row Subquery: Returns one or more rows (a list of values). These require operators that can handle multiple results, such as IN, NOT IN, ANY, or ALL.
3.Correlated Subquery: This type is dependent on the outer query and references a column from it. It is executed once for each row processed by the outer query, making it more dynamic but potentially slower than a non-correlated subquery.
4.Non-Correlated Subquery (Independent): The inner query runs completely independently of the outer query, and its results are static for the duration of the outer query's execution.

Q1.Find customers who made transactions above the overall average spending

```sql
SELECT cp.name, tl.amount
FROM transaction_logs tl
JOIN payment_cards pc ON tl.card_id = pc.card_id
JOIN customers_profile cp ON pc.customer_id = cp.customer_id
WHERE tl.amount > (SELECT AVG(amount) FROM transaction_logs)
ORDER BY tl.amount DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	amount
Emma Green	144900.00
Kim Hana	139000.00
Samar Khurana	128000.00
Oliver White	127500.00
Sophia Lewis	120000.00

Insight → Useful to spot **overspending patterns & abnormal behaviour**.

Q2. Show cards that have more fraud than normal transactions

Insight → Helps identify cards needing urgent blocking.

```sql
SELECT card_id,
       (SELECT COUNT(*) FROM transaction_logs t1
        WHERE t1.card_id = tl.card_id AND t1.is_fraud = 1) AS fraud_count
FROM transaction_logs tl
GROUP BY card_id
HAVING fraud_count > (
        SELECT COUNT(*)/2 FROM transaction_logs t2
        WHERE t2.card_id = tl.card_id
    )
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

card_id	fraud_count
4	2
17	3
28	3
32	1
33	1

Q3. Find customers who have both fraud & normal transactions

```sql
SELECT DISTINCT cp.name
FROM customers_profile cp
WHERE cp.customer_id IN (
    SELECT pc.customer_id
    FROM payment_cards pc
    JOIN transaction_logs tl ON pc.card_id = tl.card_id
    WHERE tl.is_fraud = 1)
AND cp.customer_id IN (
    SELECT pc.customer_id
    FROM payment_cards pc
    JOIN transaction_logs tl ON pc.card_id = tl.card_id
    WHERE tl.is_fraud = 0)
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Aarav Mehta
Riya Kapoor
Kabir Shah
Simran Arora
Rehan Siddiqui

Insight →
These users experienced both genuine and fraudulent activity.

Q4. Get the merchant with the highest fraud count

Insight → That merchant/platform is your primary fraud target zone

```sql
SELECT merchant, fraud_count
FROM (
    SELECT merchant, COUNT(*) AS fraud_count
    FROM transaction_logs
    WHERE is_fraud = 1
    GROUP BY merchant
) AS fraud_data
ORDER BY fraud_count DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap

merchant	fraud_count
ATM Withdrawal	3

FINAL CONCLUSION

This SQL-driven Fraud Monitoring System successfully demonstrates how financial institutions analyze card transactions to detect abnormal activity and prevent cyber-fraud.
By storing customer data, payment cards, live transactions and fraud alerts in a relational schema, the model enables multi-layered analysis using aggregations, joins, window functions and subqueries.
From the results:
•Fraud vs non-fraud distribution was clearly measurable.
•Window ranking highlighted *high-risk transactions instantly*.
•Merchant-wise fraud mapping exposed **attack-hotspots like ATM withdrawals**.
•Comparing debit vs credit card performance revealed **risk exposure by card type**.
•Subqueries helped isolate high-spend customers, repeated fraud victims & suspicious card behaviour.
Overall, the project replicates a real-world banking security environment and forms a strong foundation for advanced extensions like risk scoring dashboards and automated alert workflows. It showcases how structured data, relational logic, and query-based insights can help detect threats, improve security decisions, and reduce financial loss
It shows how SQL isn't just a database language — it is a tool for **financial risk intelligence and fraud prevention.**

THANK YOU

Fraud Detected. Future Protected.